The Journal of Open Source Software

couplr: Optimal pairing and matching via linear assignment

Gilles Colling [1¶]

1 University of Vienna, Austria ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Open Journals

Reviewers:

- @openjournals

Submitted: 01 January 1970
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

`couplr` (Colling, 2026) is an R package for optimal pairing, matching, and assignment. It solves linear assignment problems: given two sets of objects and a cost for each possible pair, choose the pairs with minimum total cost. That simple operation appears in causal inference, experimental design, task allocation, sample pairing, and image alignment.

The package provides two entry points. High-level matching functions prepare data, build distances, enforce constraints, and return analysis-ready matched data: `match_couples()` for optimal one-to-one matching, `greedy_couples()` for fast approximate matching at scale (with `sorted`, `row_best`, and priority-queue strategies), `ps_match()` for propensity-score matching, `full_match()` for variable-ratio full matching, `cem_match()` for coarsened exact matching, `subclass_match()` for propensity-score subclassification, and `cardinality_match()` for balance-constrained matching that maximizes sample size subject to standardized-difference thresholds. Match quality and robustness are assessed through `balance_diagnostics()`, `balance_table()`, and `sensitivity_analysis()` (Rosenbaum bounds), with `as_matchit()` and `bal.tab()` methods for interoperability with `MatchIt` and `cobalt`. Low-level functions such as `lap_solve()`, `lap_solve_batch()`, `lap_solve_kbest()`, and `assignment_duals()` expose the assignment solvers directly for users who already have a cost matrix; `pixel_morph()` and `pixel_morph_animate()` apply the same solvers to image alignment. This lets the same package support both statistical matching workflows and general-purpose assignment tasks.

Statement of need

Matching is a common way to make treated and control groups more comparable before downstream analysis. In practice, researchers often need more than a single nearest-neighbor match on a propensity score. They may need exact blocking by site, calipers on multiple variables, pairwise distances on observed covariates, variable-ratio full matching, replacement matching, or direct control over the assignment algorithm. Those choices are scattered across R packages and usually require users to move between data preparation, distance calculation, solver calls, and balance checks by hand.

`couplr` packages this workflow around the linear assignment problem. Users can match directly on observed covariates, add blocking and distance constraints, inspect standardized differences and distributional diagnostics, then join the matched groups for analysis. The lower-level interface remains available when the matching problem has already been reduced to a cost matrix, which matters for operations research and scientific computing use cases where rows and columns are not treated and control units.

This split is important. A causal-inference user should not need to know which solver handles rectangular matrices well, and an operations researcher should not need the package to impose a statistical matching vocabulary. `couplr` keeps those layers separate while sharing the same

tested assignment core.

State of the field

Several R packages cover parts of the matching and assignment landscape. `MatchIt` provides a widely used interface for preprocessing and matching in causal inference, supporting multiple distance metrics and matching methods with strong downstream balance support (Ho et al., 2011). `optmatch` implements optimal and full matching through network flow formulations and is often called as a back-end for distance-based matching (Hansen & Klopfer, 2006). `cobalt` supplies balance diagnostics that work across matching packages (Greifer, 2025). For the linear assignment problem itself, R users reach for `clue` (Hungarian via `solve_LSAP`), `lpSolve` (general LP), or domain-specific wrappers around individual solvers (Berkelaar & others, 2024; Hornik, 2024).

These tools split into two camps. The causal-inference packages assume a treated/control framing and route through a single fixed assignment back-end, while the solver-focused packages expose one algorithm each without the preprocessing, constraint, or diagnostic machinery a matching workflow needs. A user who wants optimal matching on a user-chosen covariate distance, with calipers, blocking, forbidden pairs, balance diagnostics, and a choice of solver, today combines three packages and writes the glue by hand. That combination is `couplr`'s first-class workflow. The package treats covariate distance as the primary entry point rather than a side option behind a propensity score, and ships 18 assignment solvers (classical dense assignment, sparse and rectangular variants, k-best, bottleneck, min-cost flow, entropy-regularized transport) implemented from scratch in C++ via Rcpp and RcppEigen, with no dependency on external R assignment libraries. Compatibility methods route matched output back into `MatchIt` and `cobalt` so existing tooling still applies. The same tested solver core serves a causal-inference user starting from a data frame and an operations-research user starting from a cost matrix.

Software design

The package has three main layers. The first layer validates inputs and converts data frames into cost matrices. It handles scaling, missing data checks, blocking, calipers, maximum-distance rules, and user-supplied weights. The second layer solves the assignment problem. All 18 solvers are implemented from scratch in C++ via Rcpp and RcppEigen, so the package adds no external solver dependency. It includes the Hungarian method (Kuhn, 1955), Jonker-Volgenant shortest augmenting paths (Jonker & Volgenant, 1987), auction algorithms (Bertsekas, 1988), cost scaling (Goldberg & Kennedy, 1995), Gabow-Tarjan scaling (Gabow & Tarjan, 1989), push-relabel min-cost flow ideas (Goldberg & Tarjan, 1988), network simplex, Ramshaw-Tarjan rectangular assignment (Ramshaw & Tarjan, 2012), and related specialized solvers. The third layer returns structured result objects with pairs, costs, diagnostics, weights, subclass information, and conversion methods for other R matching tools.

The input layer supports several distance choices and constraint types. Distances are computed from numeric covariate columns using Euclidean, Manhattan, Mahalanobis, Chebyshev, or squared-Euclidean metrics, or from a user-supplied function; propensity-score distances are available through a separate entry point. Variables can be standardized to mean zero and unit variance, rescaled to the unit interval, or robust-scaled using the median and median absolute deviation. Per-variable weights are folded into the distance by multiplying each column by the square root of its weight, so that squared differences accumulate with the intended weighting. Three constraint mechanisms reshape the cost matrix before it reaches the solver: a global `max_distance` ceiling, per-variable calipers expressed as absolute-difference thresholds, and explicit forbidden pairs supplied as index pairs. Forbidden cells receive a large finite cost rather than infinity, which keeps the matrix usable across all solvers, including those that assume

DRAFT

finite arithmetic. Blocking is built on top of these mechanisms: stratified matching either reuses an existing factor variable or constructs blocks via k-means or hierarchical clustering, and the solver is then called once per block.

The automatic solver selection in `lap_solve(method = "auto")` uses matrix shape, sparsity, cost type, and problem size to choose a method. Dense square problems can use fast dense solvers such as Jonker-Volgenant or cost scaling; sparse matrices can use sparse augmenting-path solvers; rectangular problems can avoid padding through rectangular assignment methods. Integer scaling solvers are exposed where their assumptions are met. For example, the package documents Gabow-Tarjan as an integer scaling method with theoretical dense square complexity $O(n^3 \log C)$, or $O(n^3 \log(nC))$ under the common bound that includes problem size and maximum cost. Figure 1 shows observed median solve time across problem sizes for all 18 solvers; benchmarks are reproducible via `paper/make-figure.R`.

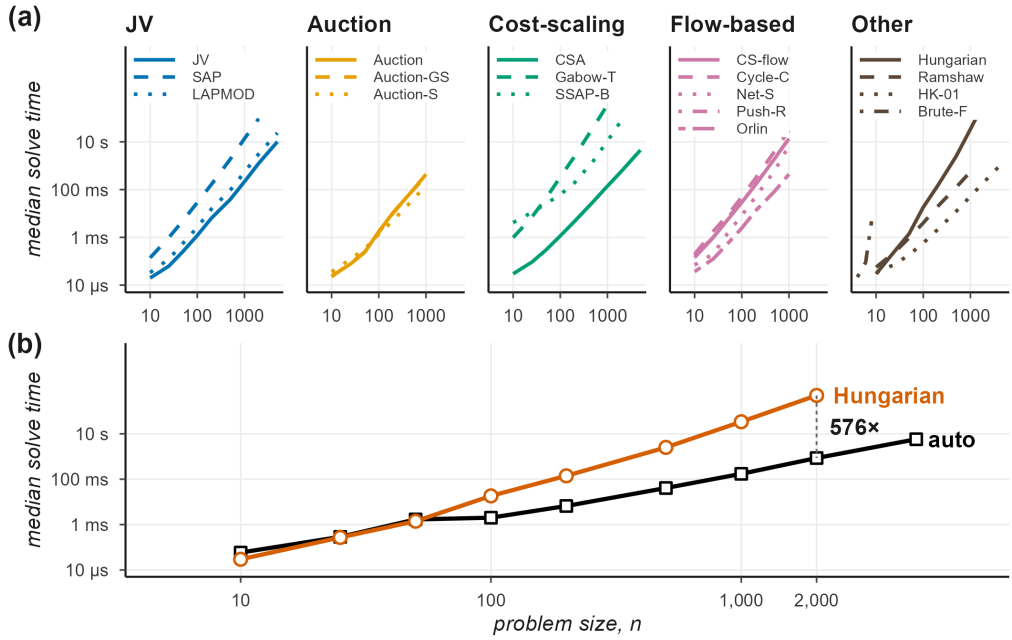

Figure 1: (a) Median wall-clock solve time versus problem size n for all 18 assignment solvers in `couplr`, arranged as five small-multiple panels grouped by algorithm family (JV / augmenting-path, Auction, Cost-scaling, Flow-based, and Other; panel headers abbreviated for space). Within each panel, individual solvers share a single family colour and are distinguished by line style; all panels share log–log axes. The *Other* panel collects single-solver families and special cases: Hungarian (classical baseline), Ramshaw–Tarjan (rectangular), HK-01 (binary $0/1$ costs, its target regime), and Bruteforce (only evaluated at $n \in \{4, 6, 8\}$). (b) The package's `auto` dispatcher (■) against the classical Hungarian baseline (●); annotated points mark the speed-up at diagnostic n. Each point is the median of 5 replicates on a single core; integer cost matrices with entries in $[1, 10{,}000]$. Reproducible from `paper/make-figure.R`.

`greedy_couples()` provides three approximate strategies (global cost sort, row-wise nearest available control, and a priority-queue variant) that share the same preprocessing and constraint machinery as `match_couples()`. The approximate path is useful when the problem size pushes optimal solvers past the user's runtime budget, and it accepts the same blocking, calipers, and per-variable weights so the two entry points are interchangeable at the call site.

The matching layer returns ordinary R objects with print, summary, plot, and join methods. Balance is assessed with `balance_diagnostics()` (standardized mean differences, variance ratios, and Kolmogorov-Smirnov statistics) and summarized through `balance_table()`; `sensitivity_analysis()` reports the critical Rosenbaum Γ at which a matched inference would no longer be significant. Blocked designs surface per-block diagnostics so imbalance

is not hidden by averaging across strata. This keeps the output usable in base R, tidyverse pipelines, and causal-inference workflows that expect matched data, weights, or subclasses.

Quickstart

The package ships with `hospital_staff`, a small example dataset used here to illustrate the core workflow on a treated/control comparison. The example runs end to end without any external data.

```r
library(couplr)

data(hospital_staff)
treated <- transform(hospital_staff$nurses_extended,   id = nurse_id)
control <- transform(hospital_staff$controls_extended, id = nurse_id)

# Optimal one-to-one matching on three covariates, with automatic
# scaling and solver selection.
m <- match_couples(
  left  = treated,
  right = control,
  vars  = c("age", "experience_years", "hourly_rate"),
  auto_scale = TRUE
)

# Balance after matching: standardized differences, variance ratios,
# and Kolmogorov-Smirnov statistics.
bal <- balance_diagnostics(
  m, treated, control,
  vars = c("age", "experience_years", "hourly_rate")
)
balance_table(bal)

# Analysis-ready output: one row per matched pair, paired covariates
# as `_left` / `_right` columns.
matched <- join_matched(m, treated, control)
```

On this dataset the matched pairs achieve absolute standardized differences below 0.15 on all three covariates. The same call accepts `calipers`, `max_distance`, `block_id`, `method`, `replace`, and `ratio` to adapt the match to harder problems; the `as_matchit()` and `bal.tab()` methods pass the result into `MatchIt` and `cobalt` for downstream analysis.

Research impact statement

`couplr` is already released on CRAN and has continuous integration across Windows, macOS, and Linux. Its documentation includes vignettes for getting started, matching workflows, algorithm selection, troubleshooting, and pixel-level morphing examples. The package is useful for experimental designs where samples must be paired before measurement, observational studies that need transparent covariate matching, and allocation problems where each object must be assigned once under costs and constraints.

AI usage disclosure

The package was developed using a modern AI-assisted developer stack. The author designed the package architecture, made the algorithm-selection choices, and wrote the implementation

DRAFT

in a TUI-first command-line workflow, using a self-hosted Qwen3-Coder-Next-REAP-48B-A3B model (mixture-of-experts coder, ~3B active per token, 4-bit MLX quantization, running on a single Apple M4 Pro) for code completion, refactoring, and boilerplate generation through Anthropic's Claude Code CLI, with requests routed to the local model and no cloud inference.

Acknowledgements

No external funding supported this work.

References

Berkelaar, M., & others. (2024). *lpSolve: Interface to lp_solve v. 5.5 to solve linear/integer programs*. https://CRAN.R-project.org/package=lpSolve

Bertsekas, D. P. (1988). The auction algorithm: A distributed relaxation method for the assignment problem. *Annals of Operations Research*, *14*, 105–123. https://doi.org/10.1007/BF02186476

Colling, G. (2026). *Couplr: Optimal pairing and matching via linear assignment*. https://doi.org/10.32614/CRAN.package.couplr

Gabow, H. N., & Tarjan, R. E. (1989). Faster scaling algorithms for network problems. *SIAM Journal on Computing*, *18*(5), 1013–1036. https://doi.org/10.1137/0218069

Goldberg, A. V., & Kennedy, R. (1995). An efficient cost scaling algorithm for the assignment problem. *Mathematical Programming*, *71*(2), 153–177. https://doi.org/10.1007/BF01585996

Goldberg, A. V., & Tarjan, R. E. (1988). A new approach to the maximum-flow problem. *Journal of the ACM*, *35*(4), 921–940. https://doi.org/10.1145/48014.61051

Greifer, N. (2025). cobalt: Covariate balance tables and plots. *Journal of Open Source Software*, *10*(109), 7610. https://doi.org/10.21105/joss.07610

Hansen, B. B., & Klopfer, S. O. (2006). Optmatch: Flexible, optimal matching for observational studies. *The R Journal*, *2*(2), 29–36. https://journal.r-project.org/archive/2006-2/RJournal_2006-2_Hansen+Klopfer.pdf

Ho, D. E., Imai, K., King, G., & Stuart, E. A. (2011). MatchIt: Nonparametric preprocessing for parametric causal inference. *Journal of Statistical Software*, *42*(8), 1–28. https://doi.org/10.18637/jss.v042.i08

Hornik, K. (2024). *Clue: Cluster ensembles*. https://CRAN.R-project.org/package=clue

Jonker, R., & Volgenant, T. (1987). A shortest augmenting path algorithm for dense and sparse linear assignment problems. *Computing*, *38*(4), 325–340. https://doi.org/10.1007/BF02278710

Kuhn, H. W. (1955). The hungarian method for the assignment problem. *Naval Research Logistics Quarterly*, *2*(1-2), 83–97. https://doi.org/10.1002/nav.3800020109

Ramshaw, L., & Tarjan, R. E. (2012). *On minimum-cost assignments in unbalanced bipartite graphs* (HPL-2012-40R1). HP Laboratories. https://www.hpl.hp.com/techreports/2012/HPL-2012-40R1.html

DRAFT